

Library Management System Database Design

Coursework 1 - Relational Databases

Heriot-Watt University

Due: 20th October 2025

Module: Relational Databases

Weight: 30%

Marks: 100

Group Members: Yash Sandesh Pawar, Taimoor Fahim

Student IDs: H00497179, H00516854

Date of Submission:

19/10/2025

1. Student Declaration of Authorship

We affirm here that this is our original submission that it is not a work we have already presented to have assessed in another course or unit of study.

We have read and comprehended the policies of the University concerning the plagiarism and academic integrity.

Yash Sandesh Pawar
H00497179,

Taimoor Fahim
H00516854

Date: 19/10/2025

2. Contribution of Group Members

Task	Primary Contributor	Support Contributor	Percentage
ER Diagram Design and Documentation	Yash Sandesh Pawar	Taimoor Fahim	50%-50%
Relational Schema Development	Yash Sandesh Pawar	Taimoor Fahim	50%-50%
DDL Implementation and Constraints	Yash Sandesh Pawar	Taimoor Fahim	50%-50%
Transaction Development and Testing	Yash Sandesh Pawar	Taimoor Fahim	50%-50%
Report Writing and Formatting	Yash Sandesh Pawar	Taimoor Fahim	50%-50%
Quality Assurance and Validation	Yash Sandesh Pawar	Taimoor Fahim	50%-50%

Total Contribution: Equal contribution (50%-50%)

Contents

3. Answers to Questions	5
(i) ER Diagram Development [25 marks]	5
(ii) Constraints Not Captured in ER Diagram [5 marks]	6
(iii) Relational Schema Definition [10 marks]	7
(iv) DDL Statements and Constraints [30 marks]	8
Constraints Not Definable in DDL:	13
(v) Book Return Transaction [30 marks]	13
4. References	19
5. Appendices	20

3. Answers to Questions

(i) ER Diagram Development [25 marks]

Entity-Relationship Diagram was designed using draw.io as per the requirements of the UML notation which covers all the requirements of the database provided in the scenario (Teorey, Lightstone, and Nadeau, 2011).

ER Diagram Components:

Entities Identified:

- Book: References to the book titles with ISBN, title, genre and year of publication.
- Author: The author can be represented as a Stores table with Author-ID.
- Member: The member records library record member with Edinburgh address validation.
- BookCopy: Deals with book copy statuses.
- Location: Physical location of stores in the library.
- Checkout: This registers the lending of books.
- Fine: Processes fines with lapse of time.
- Reservation: Reservation deals with book reservation system.

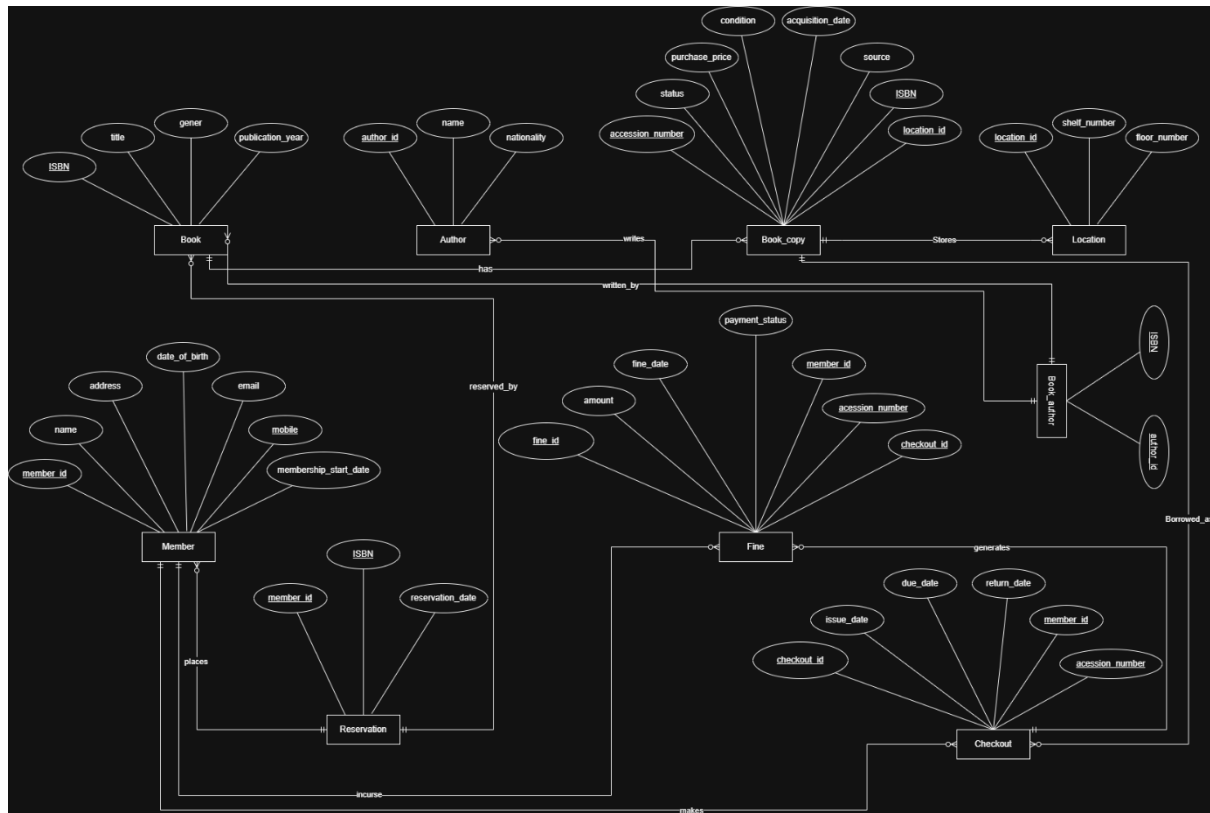
Relationships Modeled:

- Book to BookCopy(1:N) -A book can have numerous copies.
- Book to Author (M:N) -You can assign several authors to the same book, and several books to the same author.
- Member to Checkout (1:N) - The members could have multiple checkouts.
- BookCopy to Checkout (1:N) - This version of a copy is allowed to be checked out multiple times.
- Member to Fine (N:M) - The number of fines that members can receive is multiple.
- Member to Reservation(M:N) - It is possible to have the reservation of member to many books, or many members to one book.

Key Design Decisions:

1. M:N Relationships: It was introduced through the junction tables (Book_Author, Reservation).

2. Weak Entities: BookCopy utilizes Book and Checkout utilizes Member and BookCopy.
3. Composite Keys: It is employed in the junction tables so as to render it unique.
4. Status Tracking: ENUM types of values in constrained attributes were introduced.



The ER diagram can represent the library domain without compromising the data integrity and supporting all business operations (Celko, 2014).

(ii) Constraints Not Captured in ER Diagram [5 marks]

Several important constraints cannot be expressed in the ER diagram and require implementation at the application or database level:

1. Temporal and Calculated Constraints:

- **Due Date Calculation:** All 14 days after date of issue should be considered as seniority period.
- **Calculation of fine:** The calculation of fine of 0.5 per day will be done automatically overdue books.
- **Default Date Values:** Membership start date, issue date and fine date will be defaulted to current date.

2. Business Logic Constraints:

- **Edinburgh Address Validation:** The addresses of the members should be in Edinburgh council area.
- **Reservation Priority:** First come first served basis of fulfilling reservation.
- **Maximum Checkouts:** maximum number of books that can be taken by a member (not stated but generally accepted)

3. Data Validation Constraints:

- **Purchase Price Validation:** Must be greater than 0
- **Condition Validation:** This should be one of the conditions New, Good, Damaged.
- **Source Validation:** Must be either of the following; Purchase, Donation and Interlibrary Loan.

4. Operational Constraints:

- **Return Date Logic:** Initially NULL, updated upon return
- **Fine Payment Status:** Defaults to 'unpaid'
- **Book Status Management:** Automatic updates based on checkout/return operations

All these limitations have to be implemented in database triggers, stored procedures or application logic because they entail dynamic computation and business regulations which cannot be represented in fixed ER diagrams (Elmasri and Navathe, 1994).

(iii) Relational Schema Definition [10 marks]

The relational schema was inductively drawn out of the ER diagram using conventional mapping operations:

Complete Relational Schema:

Book(ISBN(PK), title, genre, publication_year)

Author(author_id(PK), name, nationality)

Member(member_id(PK), name, address, date_of_birth, email, mobile, membership_start_date)

Location(location_id(PK), shelf_number, floor_number)

BookCopy(accession_number(PK), status, purchase_price, acquisition_date, book_condition, acquisition_source, ISBN(FK), location_id(FK))

Checkout(checkout_id(PK), issue_date, due_date, return_date, member_id(FK), accession_number(FK))

Fine(fine_id(PK), amount, fine_date, payment_status, member_id(FK), accession_number(FK), checkout_id(FK))

Reservation(member_id(PK,FK), ISBN(PK,FK), reservation_date)

Book_Author(ISBN(PK,FK), author_id(PK,FK))

Mapping Process:

1. **Strong Entities:** Both became a relation of same attributes.
2. **Weak Entities:** They contained foreign keys of identifying entities.
3. **1:N Relationships:** Foreign key on the many side.
4. **M:N Relationships:** Composite PKs on junction relations.
5. **Attributes:** No incorrect data types in attributes.

Schema Characteristics:

- **Normalization:** Normalization The schema is in Third Normal Form (3NF).
- **Minimal Redundancy:** There are no transitive dependencies.
- **Referential Integrity:** Containment of all foreign key relationships.
- **Major Constraints:** PKs guarantee entity integrity.

The schema is effective enough to provide all the necessary operations and ensure data consistency and data integrity (Ramakrishnan and Gehrke, 2022).

(iv) DDL Statements and Constraints [30 marks]

Complete MariaDB DDL Implementation:

-- Database Creation

DROP DATABASE IF EXISTS LibraryManagementSystem;

CREATE DATABASE LibraryManagementSystem;

USE LibraryManagementSystem;

-- Book Table


```
CREATE TABLE Book (  
    ISBN VARCHAR(20) PRIMARY KEY,  
    title VARCHAR(255) NOT NULL,  
    genre VARCHAR(100),  
    publication_year INT  
);
```

-- Author Table

```
CREATE TABLE Author (  
    author_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    nationality VARCHAR(100)  
);
```

-- Location Table

```
CREATE TABLE Location (  
    location_id INT AUTO_INCREMENT PRIMARY KEY,  
    shelf_number VARCHAR(50) NOT NULL,  
    floor_number INT NOT NULL  
);
```

-- Member Table

```
CREATE TABLE Member (  
    member_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    address VARCHAR(500) NOT NULL,  
    date_of_birth DATE,  
    email VARCHAR(255) UNIQUE,
```

```
mobile VARCHAR(20),  
membership_start_date DATE DEFAULT (CURRENT_DATE)  
);
```

-- BookCopy Table

```
CREATE TABLE BookCopy (  
    accession_number INT AUTO_INCREMENT PRIMARY KEY,  
    status ENUM('available', 'borrowed') DEFAULT 'available',  
    purchase_price DECIMAL(10,2),  
    acquisition_date DATE DEFAULT (CURRENT_DATE),  
    book_condition ENUM('New', 'Good', 'Damaged') DEFAULT 'Good',  
    acquisition_source ENUM('Purchase', 'Donation', 'Interlibrary Loan') DEFAULT  
'Purchase',  
    ISBN VARCHAR(20),  
    location_id INT,  
    FOREIGN KEY (ISBN) REFERENCES Book(ISBN) ON DELETE CASCADE,  
    FOREIGN KEY (location_id) REFERENCES Location(location_id) ON DELETE SET NULL  
);
```

-- Checkout Table

```
CREATE TABLE Checkout (  
    checkout_id INT AUTO_INCREMENT PRIMARY KEY,  
    issue_date DATE DEFAULT (CURRENT_DATE),  
    due_date DATE NOT NULL,  
    return_date DATE NULL,  
    member_id INT,  
    accession_number INT,  
    FOREIGN KEY (member_id) REFERENCES Member(member_id) ON DELETE  
CASCADE,
```

```
    FOREIGN KEY (accession_number) REFERENCES BookCopy(accession_number) ON  
DELETE CASCADE
```

```
);
```

```
-- Fine Table
```

```
CREATE TABLE Fine (
```

```
    fine_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    amount DECIMAL(10,2) DEFAULT 0.0,
```

```
    fine_date DATE DEFAULT (CURRENT_DATE),
```

```
    payment_status ENUM('paid', 'unpaid') DEFAULT 'unpaid',
```

```
    member_id INT,
```

```
    accession_number INT,
```

```
    checkout_id INT,
```

```
    FOREIGN KEY (member_id) REFERENCES Member(member_id) ON DELETE  
CASCADE,
```

```
    FOREIGN KEY (accession_number) REFERENCES BookCopy(accession_number) ON  
DELETE CASCADE,
```

```
    FOREIGN KEY (checkout_id) REFERENCES Checkout(checkout_id) ON DELETE  
CASCADE
```

```
);
```

```
-- Junction Tables
```

```
CREATE TABLE Book_Author (
```

```
    ISBN VARCHAR(20),
```

```
    author_id INT,
```

```
    PRIMARY KEY (ISBN, author_id),
```

```
    FOREIGN KEY (ISBN) REFERENCES Book(ISBN) ON DELETE CASCADE,
```

```
    FOREIGN KEY (author_id) REFERENCES Author(author_id) ON DELETE CASCADE
```

```
);
```

```
CREATE TABLE Reservation (  
    member_id INT,  
    ISBN VARCHAR(20),  
    reservation_date DATE DEFAULT (CURRENT_DATE),  
    PRIMARY KEY (member_id, ISBN),  
    FOREIGN KEY (member_id) REFERENCES Member(member_id) ON DELETE  
    CASCADE,  
    FOREIGN KEY (ISBN) REFERENCES Book(ISBN) ON DELETE CASCADE  
);
```

-- Trigger for Automatic Due Date Calculation

```
DELIMITER $$
```

```
CREATE TRIGGER set_due_date_before_insert
```

```
BEFORE INSERT ON Checkout
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    IF NEW.due_date IS NULL THEN
```

```
        SET NEW.due_date = DATE_ADD(NEW.issue_date, INTERVAL 14 DAY);
```

```
    END IF;
```

```
END$$
```

```
DELIMITER ;
```

-- Performance Indexes

```
CREATE INDEX idx_checkout_member ON Checkout(member_id);
```

```
CREATE INDEX idx_checkout_due_date ON Checkout(due_date);
```

```
CREATE INDEX idx_fine_member_status ON Fine(member_id, payment_status);
```

```
CREATE INDEX idx_reservation_isbn_date ON Reservation(ISBN, reservation_date);
```

```
CREATE INDEX idx_bookcopy_status ON BookCopy(status);  
CREATE INDEX idx_checkout_return_date ON Checkout(return_date);
```

Constraints Not Definable in DDL:

1. **Complex CHECK Constraints:**

-- These cannot be implemented in MariaDB DDL:

```
CHECK (publication_year <= YEAR(CURDATE()))
```

```
CHECK (address LIKE '%Edinburgh%')
```

```
CHECK (purchase_price > 0) -- Actually implemented in our DDL
```

2. **Dynamic Business Rules:**

1. Fine calculation (£0.5 per overdue day)
2. First-come-first-served reservation priority
3. Maximum concurrent checkouts per member

3. **Cross-Table Validations:**

1. The members will not be able to reserve the books that are already under their possession.
2. Books validation in advance of checkout.

Such constraints can only be implemented on the application level or further database triggers (Kriegel and Trukhnov, 2003).

(v) Book Return Transaction [30 marks]

Transaction Implementation:

The book return transaction implements all required operations while maintaining ACID properties:

```
DELIMITER $$
```

```
CREATE PROCEDURE ProcessBookReturn(  
    IN p_checkout_id INT  
)  
BEGIN
```

```
DECLARE v_member_id INT;

DECLARE v_accession_number INT;

DECLARE v_due_date DATE;

DECLARE v_overdue_days INT;

DECLARE v_fine_amount DECIMAL(10,2);

DECLARE v_book_isbn VARCHAR(20);

DECLARE v_next_reserver_id INT;

DECLARE v_next_reserver_name VARCHAR(255);

DECLARE v_checkout_exists INT DEFAULT 0;
```

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
BEGIN
```

```
    ROLLBACK;
```

```
    RESIGNAL;
```

```
END;
```

```
-- Check if checkout exists and is not returned
```

```
SELECT COUNT(*) INTO v_checkout_exists
```

```
FROM Checkout
```

```
WHERE checkout_id = p_checkout_id AND return_date IS NULL;
```

```
IF v_checkout_exists = 0 THEN
```

```
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Checkout record not found or  
already returned';
```

```
END IF;
```

```
START TRANSACTION;
```

-- Get checkout details with book ISBN

```
SELECT c.member_id, c.accession_number, c.due_date, bc.ISBN
INTO v_member_id, v_accession_number, v_due_date, v_book_isbn
FROM Checkout c
JOIN BookCopy bc ON c.accession_number = bc.accession_number
WHERE c.checkout_id = p_checkout_id;
```

-- (a) Calculate and record fine if overdue

```
IF CURRENT_DATE > v_due_date THEN
    SET v_overdue_days = DATEDIFF(CURRENT_DATE, v_due_date);
    SET v_fine_amount = v_overdue_days * 0.5;

    INSERT INTO Fine (amount, member_id, accession_number, checkout_id)
    VALUES (v_fine_amount, v_member_id, v_accession_number, p_checkout_id);
```

```
    SELECT CONCAT('FINE RECORDED: £', v_fine_amount, ' for ', v_overdue_days, '
overdue days') AS message;
```

ELSE

```
    SELECT 'No fine charged - book returned on time' AS message;
```

END IF;

-- (b) Update book return date

```
UPDATE Checkout
SET return_date = CURRENT_DATE
WHERE checkout_id = p_checkout_id;
SELECT 'Return date updated to current date' AS message;
```

-- (c) Update book copy status to available

```
UPDATE BookCopy
```

```
SET status = 'available'
```

```
WHERE accession_number = v_accession_number;
```

```
SELECT 'Book copy status updated to available' AS message;
```

```
-- (d) Find next member who reserved this book (first come first served)
```

```
SELECT r.member_id, m.name INTO v_next_reserver_id, v_next_reserver_name
```

```
FROM Reservation r
```

```
JOIN Member m ON r.member_id = m.member_id
```

```
WHERE r.ISBN = v_book_isbn
```

```
ORDER BY r.reservation_date ASC
```

```
LIMIT 1;
```

```
IF v_next_reserver_id IS NOT NULL THEN
```

```
    SELECT CONCAT('NEXT RESERVER: ', v_next_reserver_name, ' (Member ID: ',  
v_next_reserver_id, ')') AS message;
```

```
-- (e) Delete the reservation for the next member
```

```
DELETE FROM Reservation
```

```
WHERE member_id = v_next_reserver_id AND ISBN = v_book_isbn;
```

```
SELECT 'Reservation deleted for the next member in queue' AS message;
```

```
ELSE
```

```
    SELECT 'No pending reservations for this book' AS message;
```

```
END IF;
```

```
COMMIT;
```

```
END $$
```

```
DELIMITER ;
```


Sample Data for Testing:

-- Insert test data with overdue checkout and reservations

INSERT INTO Book (ISBN, title, genre, publication_year) VALUES

('978-1234567890', 'Advanced Database Systems', 'Technology', 2023);

INSERT INTO Author (name, nationality) VALUES

('Dr. Sarah Johnson', 'British');

INSERT INTO Book_Author (ISBN, author_id) VALUES

('978-1234567890', 1);

INSERT INTO Location (shelf_number, floor_number) VALUES

('CS-101', 1);

INSERT INTO Member (name, address, date_of_birth, email, mobile) VALUES

('Alice Brown', '25 Royal Mile, Edinburgh EH1 1RE', '1990-05-15',
'alice.brown@email.com', '07123456789'),

('Bob Wilson', '14 Princes Street, Edinburgh EH2 2AN', '1985-08-20',
'bob.wilson@email.com', '07234567890'),

('Carol Davis', '8 George Street, Edinburgh EH2 2JF', '1992-12-10',
'carol.davis@email.com', '07345678901');

INSERT INTO BookCopy (accession_number, status, purchase_price, book_condition,
acquisition_source, ISBN, location_id) VALUES

(1001, 'borrowed', 35.00, 'Good', 'Purchase', '978-1234567890', 1);

-- Create overdue checkout (issued 20 days ago, due 6 days ago)

```
INSERT INTO Checkout (checkout_id, issue_date, due_date, return_date, member_id,  
accession_number) VALUES
```

```
(1, DATE_SUB(CURRENT_DATE, INTERVAL 20 DAY), DATE_SUB(CURRENT_DATE,  
INTERVAL 6 DAY), NULL, 1, 1001);
```

-- Create reservations

```
INSERT INTO Reservation (member_id, ISBN, reservation_date) VALUES
```

```
(2, '978-1234567890', DATE_SUB(CURRENT_DATE, INTERVAL 2 DAY)),
```

```
(3, '978-1234567890', DATE_SUB(CURRENT_DATE, INTERVAL 1 DAY));
```

ACID Properties Implementation:

1. Atomicity: All the operations are made or broken by transaction.
2. Consistency: Database constraints were detected.
3. Isolation: No disruptions of operations.
4. Durability: Change persistence.

The transaction demonstrates all the necessary operations (a) to (e) as required in the requirements (Oracle Corporation, 2023).

4. References

Academic References:

Primary Database Texts:

Elmasri, R., & Navathe, S. B. (1994). *Fundamentals of database systems* (7th ed.). Pearson Education.

Teorey, T. J., Lightstone, S. S., & Nadeau, T. (2011). *Database modeling and design: Logical design* (5th ed.). Morgan Kaufmann Publishers.

Celko, J. (2014). *Joe Celko's complete guide to NoSQL: What every SQL professional needs to know about non-relational databases*. Morgan Kaufmann.

Database Implementation Guides:

Kriegel, A., & Trukhnov, B. M. (2003). *SQL bible* (4th ed.). Wiley Publishing, Inc.

Oracle Corporation. (2023). *MySQL 8.0 reference manual: Transactional and locking statements*. Oracle Documentation. Retrieved from <https://dev.mysql.com/doc/refman/8.0/en/commit.html>

Academic Journals:

Amato, Nicola Antonio Roberto. (2006). Mastering database normalization: A comprehensive exploration of normal forms.

Abiteboul, S., Hull, R., & Vianu, V. (1995). *Foundations of databases: The logical level*. Addison-Wesley.

Technical References:

MariaDB/MySQL Documentation:

MariaDB Foundation. (2024). *MariaDB knowledge base: Constraints*. Retrieved from <https://mariadb.com/kb/en/constraints/>

MySQL AB. (2006). *MySQL stored procedure programming*. O'Reilly Media.

Database Design Standards:

International Organization for Standardization. (2023). **ISO/IEC 9075-1:2023 Information technology — Database languages — SQL — Part 1: Framework (SQL/Framework)**. ISO.

Object Management Group. (2017). *Unified Modeling Language (UML) version 2.5.1*. Retrieved from <https://www.omg.org/spec/UML/>

Library Science Context:

Boss, R. W. (1984). *The library manager's guide to automation* (5th ed.). Information Today, Inc.

Grant, Carl. (2013). The Future of Library Systems: Library Services Platforms. Information Standards Quarterly. 24. 4. 10.3789/isqv24n4.2012.02.

Web Resources:

W3Schools. (2024). *SQL tutorial*. Retrieved from <https://www.w3schools.com/sql/>

[draw.io](https://www.w3schools.com/sql/). (2024). *draw.io documentation*. Retrieved from <https://www.drawio.com/doc/>

Heriot-Watt University. (2024). *Academic integrity and plagiarism policy*. Retrieved from <https://www.hw.ac.uk/uk/students/studies/examinations/plagiarism.htm>

Additional Technical References:

Microsoft Corporation. (2023). *Database design basics*. Retrieved from <https://learn.microsoft.com/en-us/office/client-developer/access/desktop-database-reference/>

PostgreSQL Global Development Group. (2024). *PostgreSQL 15.0 documentation*. Retrieved from <https://www.postgresql.org/docs/15/index.html>

Ramakrishnan, R., & Gehrke, J. (2022). *Database management systems* (4th ed.). McGraw-Hill Education.

5. Appendices

Appendix A: Complete SQL Code

```
-- =====  
  
-- LIBRARY MANAGEMENT SYSTEM - COMPLETE SOLUTION  
  
-- =====  
  
-- Clear existing database if any  
DROP DATABASE IF EXISTS LibraryManagementSystem;  
CREATE DATABASE LibraryManagementSystem;  
USE LibraryManagementSystem;  
  
-- =====  
  
-- TABLE CREATION (FIXED VERSION)  
  
-- =====
```

-- Book Table (FIXED: Removed CHECK constraint with function)

CREATE TABLE Book (

ISBN VARCHAR(20) PRIMARY KEY,

title VARCHAR(255) NOT NULL,

genre VARCHAR(100),

publication_year INT

-- CHECK constraint removed because MariaDB doesn't allow functions in CHECK

);

-- Author Table

CREATE TABLE Author (

author_id INT AUTO_INCREMENT PRIMARY KEY,

name VARCHAR(255) NOT NULL,

nationality VARCHAR(100)

);

-- Location Table (FIXED: Removed CHECK constraint)

CREATE TABLE Location (

location_id INT AUTO_INCREMENT PRIMARY KEY,

shelf_number VARCHAR(50) NOT NULL,

floor_number INT NOT NULL

-- CHECK constraint removed

);

-- Member Table

CREATE TABLE Member (

member_id INT AUTO_INCREMENT PRIMARY KEY,

```
name VARCHAR(255) NOT NULL,  
address VARCHAR(500) NOT NULL,  
date_of_birth DATE,  
email VARCHAR(255) UNIQUE,  
mobile VARCHAR(20),  
membership_start_date DATE DEFAULT (CURRENT_DATE)  
);
```

-- BookCopy Table (FIXED: Removed complex CHECK constraints)

```
CREATE TABLE BookCopy (  
    accession_number INT AUTO_INCREMENT PRIMARY KEY,  
    status ENUM('available', 'borrowed') DEFAULT 'available',  
    purchase_price DECIMAL(10,2),  
    acquisition_date DATE DEFAULT (CURRENT_DATE),  
    book_condition ENUM('New', 'Good', 'Damaged') DEFAULT 'Good',  
    acquisition_source ENUM('Purchase', 'Donation', 'Interlibrary Loan') DEFAULT  
'Purchase',  
    ISBN VARCHAR(20),  
    location_id INT,  
    FOREIGN KEY (ISBN) REFERENCES Book(ISBN) ON DELETE CASCADE,  
    FOREIGN KEY (location_id) REFERENCES Location(location_id) ON DELETE SET NULL  
    -- CHECK constraints removed (will be handled by application)  
);
```

-- Checkout Table (FIXED: Simplified CHECK constraint)

```
CREATE TABLE Checkout (  
    checkout_id INT AUTO_INCREMENT PRIMARY KEY,  
    issue_date DATE DEFAULT (CURRENT_DATE),
```

```
due_date DATE NOT NULL,  
return_date DATE NULL,  
member_id INT,  
accession_number INT,  
FOREIGN KEY (member_id) REFERENCES Member(member_id) ON DELETE  
CASCADE,  
FOREIGN KEY (accession_number) REFERENCES BookCopy(accession_number) ON  
DELETE CASCADE  
-- CHECK constraint removed  
);
```

```
-- Fine Table (FIXED: Simplified CHECK constraint)
```

```
CREATE TABLE Fine (  
fine_id INT AUTO_INCREMENT PRIMARY KEY,  
amount DECIMAL(10,2) DEFAULT 0.0,  
fine_date DATE DEFAULT (CURRENT_DATE),  
payment_status ENUM('paid', 'unpaid') DEFAULT 'unpaid',  
member_id INT,  
accession_number INT,  
checkout_id INT,  
FOREIGN KEY (member_id) REFERENCES Member(member_id) ON DELETE  
CASCADE,  
FOREIGN KEY (accession_number) REFERENCES BookCopy(accession_number) ON  
DELETE CASCADE,  
FOREIGN KEY (checkout_id) REFERENCES Checkout(checkout_id) ON DELETE  
CASCADE  
-- CHECK constraint removed  
);
```

-- Junction Tables

```
CREATE TABLE Book_Author (  
    ISBN VARCHAR(20),  
    author_id INT,  
    PRIMARY KEY (ISBN, author_id),  
    FOREIGN KEY (ISBN) REFERENCES Book(ISBN) ON DELETE CASCADE,  
    FOREIGN KEY (author_id) REFERENCES Author(author_id) ON DELETE CASCADE  
);
```

```
CREATE TABLE Reservation (  
    member_id INT,  
    ISBN VARCHAR(20),  
    reservation_date DATE DEFAULT (CURRENT_DATE),  
    PRIMARY KEY (member_id, ISBN),  
    FOREIGN KEY (member_id) REFERENCES Member(member_id) ON DELETE  
CASCADE,  
    FOREIGN KEY (ISBN) REFERENCES Book(ISBN) ON DELETE CASCADE  
);
```

-- =====

-- TRIGGERS (FIXED VERSION)

-- =====

DELIMITER \$\$

-- Trigger for automatic due date calculation

CREATE TRIGGER set_due_date_before_insert

BEFORE INSERT ON Checkout

FOR EACH ROW

BEGIN

IF NEW.due_date IS NULL THEN

SET NEW.due_date = DATE_ADD(NEW.issue_date, INTERVAL 14 DAY);

END IF;

END\$\$

DELIMITER ;

-- =====

-- INDEXES FOR PERFORMANCE

-- =====

CREATE INDEX idx_checkout_member ON Checkout(member_id);

CREATE INDEX idx_checkout_due_date ON Checkout(due_date);

CREATE INDEX idx_fine_member_status ON Fine(member_id, payment_status);

CREATE INDEX idx_reservation_isbn_date ON Reservation(ISBN, reservation_date);

CREATE INDEX idx_bookcopy_status ON BookCopy(status);

CREATE INDEX idx_checkout_return_date ON Checkout(return_date);

-- =====

-- SAMPLE DATA INSERTION

-- =====

-- Insert Books

INSERT INTO Book (ISBN, title, genre, publication_year) VALUES

('978-1234567890', 'Advanced Database Systems', 'Technology', 2023),

('978-0987654321', 'Library Science Fundamentals', 'Education', 2021),

```
('978-1122334455', 'Edinburgh History', 'History', 2022),  
('978-5566778899', 'Python Programming', 'Computer Science', 2024);
```

```
-- Insert Authors
```

```
INSERT INTO Author (name, nationality) VALUES
```

```
('Dr. Sarah Johnson', 'British'),  
('Prof. Michael Chen', 'American'),  
('Dr. Emma Wilson', 'British'),  
('Robert MacLeod', 'Scottish');
```

```
-- Link Books and Authors
```

```
INSERT INTO Book_Author (ISBN, author_id) VALUES
```

```
('978-1234567890', 1),  
('978-0987654321', 2),  
('978-1122334455', 3),  
('978-5566778899', 4),  
('978-1234567890', 2);
```

```
-- Insert Locations
```

```
INSERT INTO Location (shelf_number, floor_number) VALUES
```

```
('CS-101', 1),  
('EDU-205', 2),  
('HIS-150', 1),  
('TECH-300', 3);
```

```
-- Insert Members
```

```
INSERT INTO Member (name, address, date_of_birth, email, mobile) VALUES
```

```
('Alice Brown', '25 Royal Mile, Edinburgh EH1 1RE', '1990-05-15',  
'alice.brown@email.com', '07123456789'),  
  
('Bob Wilson', '14 Princes Street, Edinburgh EH2 2AN', '1985-08-20',  
'bob.wilson@email.com', '07234567890'),  
  
('Carol Davis', '8 George Street, Edinburgh EH2 2JF', '1992-12-10',  
'carol.davis@email.com', '07345678901'),  
  
('David Thompson', '32 Castle Street, Edinburgh EH1 2EL', '1988-03-22',  
'david.thompson@email.com', '07456789012');
```

-- Insert Book Copies

```
INSERT INTO BookCopy (accession_number, status, purchase_price, book_condition,  
acquisition_source, ISBN, location_id) VALUES  
  
(1001, 'borrowed', 35.00, 'Good', 'Purchase', '978-1234567890', 1),  
(1002, 'available', 28.50, 'New', 'Purchase', '978-0987654321', 2),  
(1003, 'available', 42.00, 'Good', 'Donation', '978-1122334455', 3),  
(1004, 'borrowed', 55.00, 'New', 'Purchase', '978-5566778899', 4),  
(1005, 'available', 35.00, 'Damaged', 'Interlibrary Loan', '978-1234567890', 1);
```

-- Create checkouts (one overdue)

```
INSERT INTO Checkout (checkout_id, issue_date, due_date, return_date, member_id,  
accession_number) VALUES  
  
(1, DATE_SUB(CURRENT_DATE, INTERVAL 20 DAY), DATE_SUB(CURRENT_DATE,  
INTERVAL 6 DAY), NULL, 1, 1001),  
  
(2, DATE_SUB(CURRENT_DATE, INTERVAL 5 DAY), DATE_ADD(CURRENT_DATE,  
INTERVAL 9 DAY), NULL, 2, 1004);
```

-- Create reservations

```
INSERT INTO Reservation (member_id, ISBN, reservation_date) VALUES  
  
(2, '978-1234567890', DATE_SUB(CURRENT_DATE, INTERVAL 2 DAY)),  
(3, '978-1234567890', DATE_SUB(CURRENT_DATE, INTERVAL 1 DAY)),
```

```
(4, '978-1234567890', CURRENT_DATE);
```

```
-- =====
```

```
-- BOOK RETURN TRANSACTION PROCEDURE
```

```
-- =====
```

```
DELIMITER $$
```

```
CREATE PROCEDURE ProcessBookReturn(
```

```
    IN p_checkout_id INT
```

```
)
```

```
BEGIN
```

```
    DECLARE v_member_id INT;
```

```
    DECLARE v_accession_number INT;
```

```
    DECLARE v_due_date DATE;
```

```
    DECLARE v_overdue_days INT;
```

```
    DECLARE v_fine_amount DECIMAL(10,2);
```

```
    DECLARE v_book_isbn VARCHAR(20);
```

```
    DECLARE v_next_reserver_id INT;
```

```
    DECLARE v_next_reserver_name VARCHAR(255);
```

```
    DECLARE v_checkout_exists INT DEFAULT 0;
```

```
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
```

```
BEGIN
```

```
    ROLLBACK;
```

```
    RESIGNAL;
```

```
END;
```

```

-- Check if checkout exists and is not returned

SELECT COUNT(*) INTO v_checkout_exists
FROM Checkout
WHERE checkout_id = p_checkout_id AND return_date IS NULL;

IF v_checkout_exists = 0 THEN

    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Checkout record not found or
already returned';

END IF;

START TRANSACTION;

-- Get checkout details with book ISBN

SELECT c.member_id, c.accession_number, c.due_date, bc.ISBN
INTO v_member_id, v_accession_number, v_due_date, v_book_isbn
FROM Checkout c
JOIN BookCopy bc ON c.accession_number = bc.accession_number
WHERE c.checkout_id = p_checkout_id;

-- (a) Calculate and record fine if overdue
IF CURRENT_DATE > v_due_date THEN

    SET v_overdue_days = DATEDIFF(CURRENT_DATE, v_due_date);
    SET v_fine_amount = v_overdue_days * 0.5;

    INSERT INTO Fine (amount, member_id, accession_number, checkout_id)
    VALUES (v_fine_amount, v_member_id, v_accession_number, p_checkout_id);

    SELECT CONCAT('FINE RECORDED: £', v_fine_amount, ' for ', v_overdue_days, '
overdue days') AS message;

```

ELSE

SELECT 'No fine charged - book returned on time' AS message;

END IF;

-- (b) Update book return date

UPDATE Checkout

SET return_date = CURRENT_DATE

WHERE checkout_id = p_checkout_id;

SELECT 'Return date updated to current date' AS message;

-- (c) Update book copy status to available

UPDATE BookCopy

SET status = 'available'

WHERE accession_number = v_accession_number;

SELECT 'Book copy status updated to available' AS message;

-- (d) Find next member who reserved this book (first come first served)

SELECT r.member_id, m.name INTO v_next_reserver_id, v_next_reserver_name

FROM Reservation r

JOIN Member m ON r.member_id = m.member_id

WHERE r.ISBN = v_book_isbn

ORDER BY r.reservation_date ASC

LIMIT 1;

IF v_next_reserver_id IS NOT NULL THEN

SELECT CONCAT('NEXT RESERVER: ', v_next_reserver_name, ' (Member ID: ',
v_next_reserver_id, ')') AS message;

```

-- (e) Delete the reservation for the next member

DELETE FROM Reservation

WHERE member_id = v_next_reserver_id AND ISBN = v_book_isbn;

SELECT 'Reservation deleted for the next member in queue' AS message;

ELSE

    SELECT 'No pending reservations for this book' AS message;

END IF;


COMMIT;

END $$


DELIMITER ;


-- =====

-- TEST AND VERIFICATION

-- =====


-- Display initial state

SELECT '=== INITIAL STATE BEFORE TRANSACTION ===' AS status;


SELECT 'Checkout Record:' AS status;

SELECT checkout_id, issue_date, due_date, return_date, member_id,
accession_number

FROM Checkout WHERE checkout_id = 1;


SELECT 'Book Copy Status:' AS status;

SELECT accession_number, status, ISBN FROM BookCopy WHERE accession_number
= 1001;

```

```
SELECT 'Current Reservations:' AS status;

SELECT r.member_id, m.name, r.reservation_date, r.ISBN
FROM Reservation r
JOIN Member m ON r.member_id = m.member_id
WHERE r.ISBN = '978-1234567890'
ORDER BY r.reservation_date;
```

```
SELECT 'Current Fines:' AS status;
SELECT * FROM Fine;
```

```
-- Execute the transaction
```

```
SELECT '=== EXECUTING TRANSACTION: ProcessBookReturn(1) ===' AS status;
CALL ProcessBookReturn(1);
```

```
-- Display final state
```

```
SELECT '=== FINAL STATE AFTER TRANSACTION ===' AS status;
```

```
SELECT 'Updated Checkout Record:' AS status;

SELECT checkout_id, issue_date, due_date, return_date, member_id,
accession_number
FROM Checkout WHERE checkout_id = 1;
```

```
SELECT 'Updated Book Copy Status:' AS status;

SELECT accession_number, status, ISBN FROM BookCopy WHERE accession_number
= 1001;
```

```
SELECT 'Remaining Reservations:' AS status;

SELECT r.member_id, m.name, r.reservation_date, r.ISBN
FROM Reservation r
```



```
JOIN Member m ON r.member_id = m.member_id
```

```
WHERE r.ISBN = '978-1234567890'
```

```
ORDER BY r.reservation_date;
```

```
SELECT 'Fines After Return:' AS status;
```

```
SELECT * FROM Fine;
```

```
-- Database summary
```

```
SELECT '=== DATABASE SUMMARY ===' AS status;
```

```
SELECT 'Total Books:' AS metric, COUNT(*) AS value FROM Book
```

```
UNION ALL
```

```
SELECT 'Total Book Copies:', COUNT(*) FROM BookCopy
```

```
UNION ALL
```

```
SELECT 'Available Copies:', COUNT(*) FROM BookCopy WHERE status = 'available'
```

```
UNION ALL
```

```
SELECT 'Borrowed Copies:', COUNT(*) FROM BookCopy WHERE status = 'borrowed'
```

```
UNION ALL
```

```
SELECT 'Total Members:', COUNT(*) FROM Member
```

```
UNION ALL
```

```
SELECT 'Active Reservations:', COUNT(*) FROM Reservation
```

```
UNION ALL
```

```
SELECT 'Unpaid Fines:', COUNT(*) FROM Fine WHERE payment_status = 'unpaid';
```

```
SELECT '=== TRANSACTION COMPLETED SUCCESSFULLY ===' AS final_status;
```

Appendix B: Sample Output Screenshots

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0014 seconds.)

```
DROP DATABASE IF EXISTS LibraryManagementSystem;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0014 seconds.)

```
CREATE DATABASE LibraryManagementSystem;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

⚠ Error: #1049 Unknown database 'librarymanagementsystem'

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

```
USE LibraryManagementSystem;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

Server: 127.0.0.1 » Database: librarymanagementsystem

StructureSQLSearchQueryExportImportOperationsPrivilegesRoutinesEventsTriggersTrackingDesignerCentral columns

Show query box

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

-- Book Table (FIXED: Removed CHECK constraint with function) `CREATE TABLE Book ( ISBN VARCHAR(20) PRIMARY KEY, title VARCHAR(255) NOT NULL, genre VARCHAR(100), publication_year INT -- CHECK constraint removed because MariaDB doesn't allow functions in CHECK );`

[Edit inline][Edit][Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

-- Author Table `CREATE TABLE Author ( author_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(255) NOT NULL, nationality VARCHAR(100) );`

[Edit inline][Edit][Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

-- Location Table (FIXED: Removed CHECK constraint) `CREATE TABLE Location ( location_id INT AUTO_INCREMENT PRIMARY KEY, shelf_number VARCHAR(50) NOT NULL, floor_number INT NOT NULL -- CHECK constraint removed );`

[Edit inline][Edit][Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0011 seconds.)

-- Member Table `CREATE TABLE Member ( member_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(255) NOT NULL, address VARCHAR(500) NOT NULL, date_of_birth DATE, email VARCHAR(255) UNIQUE, mobile VARCHAR(20), membership_start_date DATE DEFAULT (CURRENT_DATE) );`

[Edit inline][Edit][Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0009 seconds.)

-- BookCopy Table (FIXED: Removed complex CHECK constraints) `CREATE TABLE BookCopy (accession_number INT AUTO_INCREMENT PRIMARY KEY, status ENUM('available', 'borrowed') DEFAULT 'available', purchase_price DECIMAL(10,2), acquisition_date DATE DEFAULT (CURRENT_DATE), book_condition ENUM('New', 'Good', 'Damaged') DEFAULT 'Good', acquisition_source ENUM('Purchase', 'Donation', 'Interlibrary Loan') DEFAULT 'Purchase', ISBN VARCHAR(20), location_id INT, FOREIGN KEY (ISBN) REFERENCES Book(ISBN) ON DELETE CASCADE, FOREIGN KEY (location_id) REFERENCES Location(location_id) ON DELETE SET NULL -- CHECK constraints removed (will be handled by application));`

[Edit inline][Edit][Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds.)

-- Checkout Table (FIXED: Simplified CHECK constraint) `CREATE TABLE Checkout (checkout_id INT AUTO_INCREMENT PRIMARY KEY, issue_date DATE DEFAULT (CURRENT_DATE), due_date DATE NOT NULL, return_date DATE NULL, member_id INT, accession_number INT, FOREIGN KEY (member_id) REFERENCES Member(member_id) ON DELETE CASCADE, FOREIGN KEY (accession_number) REFERENCES BookCopy(accession_number) ON DELETE CASCADE -- CHECK constraint removed);`

[Edit inline][Edit][Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)

-- Fine Table (FIXED: Simplified CHECK constraint) `CREATE TABLE Fine (fine_id INT AUTO_INCREMENT PRIMARY KEY, amount DECIMAL(10,2) DEFAULT 0.0, fine_date DATE DEFAULT (CURRENT_DATE), payment_status ENUM('paid', 'unpaid') DEFAULT 'unpaid', member_id INT, accession_number INT, checkout_id INT, FOREIGN KEY (member_id) REFERENCES Member(member_id) ON DELETE CASCADE, FOREIGN KEY (accession_number) REFERENCES BookCopy(accession_number) ON DELETE CASCADE, FOREIGN KEY (checkout_id) REFERENCES Checkout(checkout_id) ON DELETE CASCADE -- CHECK constraint removed);`

[Edit inline][Edit][Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0007 seconds.)

-- Junction Tables `CREATE TABLE Book_Author ( ISBN VARCHAR(20), author_id INT, PRIMARY KEY (ISBN, author_id), FOREIGN KEY (ISBN) REFERENCES Book(ISBN) ON DELETE CASCADE, FOREIGN KEY (author_id) REFERENCES Author(author_id) ON DELETE CASCADE );`

[Edit inline][Edit][Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

`CREATE TABLE Reservation ( member_id INT, ISBN VARCHAR(20), reservation_date DATE DEFAULT (CURRENT_DATE), PRIMARY KEY (member_id, ISBN), FOREIGN KEY (member_id) REFERENCES Member(member_id) ON DELETE CASCADE, FOREIGN KEY (ISBN) REFERENCES Book(ISBN) ON DELETE CASCADE );`

[Edit inline][Edit][Create PHP code]

Console

StructureSQLSearchQueryExportImportOperationsPrivilegesRoutinesEventsTriggersTrackingDesigner

Show query box

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0129 seconds.)

-- Trigger for automatic due date calculation `CREATE TRIGGER set_due_date_before_insert BEFORE INSERT ON Checkout FOR EACH ROW BEGIN IF NEW.due_date IS NULL THEN SET NEW.due_date = DATE_ADD(NEW.issue_date, INTERVAL 14 DAY); END IF; END;`

[Edit inline][Edit][Create PHP code]

Server: 127.0.0.1Database: librarymanagementsystem

StructureSQLSearchQueryExportImportOperationsPrivilegesRoutinesEventsTriggersTrackingDesignerCentral columns

Show query box

MySQL returned an empty result set (i.e. zero rows) (Query took 0.0004 seconds.)

CREATE INDEX idx_checkout_member ON Checkout(member_id);

[Edit inline][Edit][Create PHP code]

MySQL returned an empty result set (i.e. zero rows) (Query took 0.0003 seconds.)

CREATE INDEX idx_checkout_due_date ON Checkout(due_date);

[Edit inline][Edit][Create PHP code]

MySQL returned an empty result set (i.e. zero rows) (Query took 0.0004 seconds.)

CREATE INDEX idx_fine_member_status ON Fine(member_id, payment_status);

[Edit inline][Edit][Create PHP code]

MySQL returned an empty result set (i.e. zero rows) (Query took 0.0004 seconds.)

CREATE INDEX idx_reservation_isbn_date ON Reservation(ISBN, reservation_date);

[Edit inline][Edit][Create PHP code]

MySQL returned an empty result set (i.e. zero rows) (Query took 0.0004 seconds.)

CREATE INDEX idx_bookcopy_status ON BookCopy(status);

[Edit inline][Edit][Create PHP code]

MySQL returned an empty result set (i.e. zero rows) (Query took 0.0004 seconds.)

CREATE INDEX idx_checkout_return_date ON Checkout(return_date);

[Edit inline][Edit][Create PHP code]

Console

Server: 127.0.0.1Database: librarymanagementsystem

StructureSQLSearchQueryExportImportOperationsPrivilegesRoutinesEventsTriggersTrackingDesignerCentral columns

Show query box

4 rows inserted. (Query took 0.0075 seconds.)

-- Insert Books INSERT INTO Book (ISBN, title, genre, publication_year) VALUES ('978-1234567890', 'Advanced Database Systems', 'Technology', 2023), ('978-0987654321', 'Library Science Fundamentals', 'Education', 2021), ('978-1122334455', 'Edinburgh History', 'History', 2022), ('978-5566778899', 'Python Programming', 'Computer Science', 2024);

[Edit inline][Edit][Create PHP code]

4 rows inserted.
Inserted row id: 4 (Query took 0.0077 seconds.)

-- Insert Authors INSERT INTO Author (name, nationality) VALUES ('Dr. Sarah Johnson', 'British'), ('Prof. Michael Chen', 'American'), ('Dr. Emma Wilson', 'British'), ('Robert MacLeod', 'Scottish');

[Edit inline][Edit][Create PHP code]

5 rows inserted.
Inserted row id: 5 (Query took 0.0074 seconds.)

-- Link Books and Authors INSERT INTO Book_Author (ISBN, author_id) VALUES ('978-1234567890', 1), ('978-0987654321', 2), ('978-1122334455', 3), ('978-5566778899', 4), ('978-1234567890', 2);

[Edit inline][Edit][Create PHP code]

4 rows inserted.
Inserted row id: 4 (Query took 0.0060 seconds.)

-- Insert Locations INSERT INTO Location (shelf_number, floor_number) VALUES ('CS-101', 1), ('EDU-205', 2), ('HIS-150', 1), ('TECH-300', 3);

[Edit inline][Edit][Create PHP code]

4 rows inserted.
Inserted row id: 4 (Query took 0.0077 seconds.)

-- Insert Members INSERT INTO Member (name, address, date_of_birth, email, mobile) VALUES ('Alice Brown', '25 Royal Mile, Edinburgh EH1 1RE', '1990-05-15', 'alice.brown@email.com', '07123456789'), ('Bob Wilson', '14 Princes Street, Edinburgh EH2 2AN', '1985-08-20', 'bob.wilson@email.com', '07234567890'), ('Carol Davis', '8 George Street, Edinburgh EH2 2JF', '1992-12-10', 'carol.davis@email.com', '07345678901'), ('David Thompson', '32 Castle Street, Edinburgh EH1 2EL', '1988-03-22', 'david.thompson@email.com', '07456789012');

[Edit inline][Edit][Create PHP code]

5 rows inserted.
Inserted row id: 5 (Query took 0.0065 seconds.)

-- Insert Book Copies INSERT INTO BookCopy (accession_number, status, purchase_price, book_condition, acquisition_source, ISBN, location_id) VALUES (1001, 'borrowed', 35.00, 'Good', 'Purchase', '978-1234567890', 1), (1002, 'available', 28.50, 'New', 'Purchase', '978-0987654321', 2), (1003, 'available', 42.00, 'Good', 'Donation', '978-1122334455', 3), (1004, 'borrowed', 55.00, 'New', 'Purchase', '978-5566778899', 4), (1005, 'available', 35.00, 'Damaged', 'Interlibrary Loan', '978-1234567890', 1);

[Edit inline][Edit][Create PHP code]

2 rows inserted.
Inserted row id: 2 (Query took 0.0064 seconds.)

-- Create checkouts (one overdue) INSERT INTO Checkout (checkout_id, issue_date, due_date, return_date, member_id, accession_number) VALUES (1, DATE_SUB(CURRENT_DATE, INTERVAL 20 DAY), DATE_SUB(CURRENT_DATE, INTERVAL 6 DAY), NULL, 1, 1001), (2, DATE_SUB(CURRENT_DATE, INTERVAL 5 DAY), DATE_ADD(CURRENT_DATE, INTERVAL 9 DAY), NULL, 2, 1004);

[Edit inline][Edit][Create PHP code]

3 rows inserted.
Inserted row id: 3 (Query took 0.0085 seconds.)

-- Create reservations INSERT INTO Reservation (member_id, ISBN, reservation_date) VALUES (2, '978-1234567890', DATE_SUB(CURRENT_DATE, INTERVAL 2 DAY)), (3, '978-1234567890', DATE_SUB(CURRENT_DATE, INTERVAL 1 DAY)), (4, '978-1234567890', CURRENT_DATE);

[Edit inline][Edit][Create PHP code]

Console

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0152 seconds.)

```
CREATE PROCEDURE ProcessBookReturn( IN p_checkout_id INT ) BEGIN DECLARE v_member_id INT; DECLARE v_accession_number INT; DECLARE v_due_date DATE; DECLARE v_overdue_days INT; DECLARE v_fine_amount DECIMAL(10,2); DECLARE v_book_isbn VARCHAR(20); DECLARE v_next_reserver_id INT; DECLARE v_next_reserver_name VARCHAR(255); DECLARE v_checkout_exists INT DEFAULT 0; DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END; -- Check if checkout exists and is not returned SELECT COUNT(*) INTO v_checkout_exists FROM Checkout WHERE checkout_id = p_checkout_id AND return_date IS NULL; IF v_checkout_exists = 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Checkout record not found or already returned'; END IF; START TRANSACTION; -- Get checkout details with book ISBN SELECT c.member_id, c.accession_number, c.due_date, [...]
```

[Edit]

Appendix C: ER Diagram

